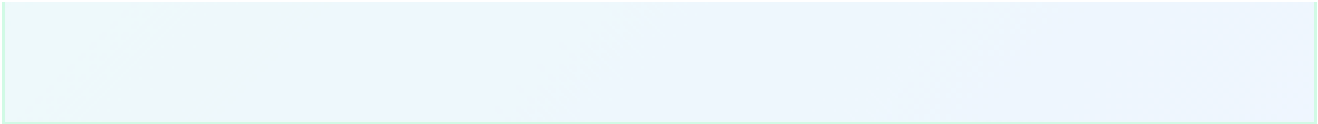


Sarowth Geliřtirici El Kitabı

Kiřisel büte, canlı haber ve ürün agentlarıyla alıřan Türke karar destek sistemini sıfırdan geliřtirmek için kapsamlı teknik rehber.

Bu belge, projeyi yapay zeka kullanmadan yeniden geliřtirmek isteyen bir geliřtiricinin mimariyi, algoritmaları, teknolojileri ve mühendislik kararlarını öğrenmesi için hazırlanmıştır.



İçindekiler

1. Projenin amacı ve ürün mantığı
2. Kullanılan teknolojiler
3. Sistem mimarisi
4. Veritabanı ve auth tasarımı
5. Bütçe modülü ve karar motoru
6. Chat asistanı ve Gemini entegrasyonu
7. Agent mimarisi
8. Haber, ürün ve fuzzy arama algoritmaları
9. Yenileme, tekrar önleme ve para birimi dönüşümü
10. Sıfırdan geliştirme planı

1. Projenin Amacı

Sarowth, kişisel bütçe yönetimini canlı haber, ürün trendi ve tedarik sinyalleriyle birleştiren web tabanlı bir karar destek sistemidir. Kullanıcı gelir, gider ve birikim kayıtlarını girer. Sistem bu kayıtları kullanarak serbest bütçe, risk bandı ve güvenli test bütçesi hesaplar.

Bu sistemin farkı, yapay zeka cevabını merkeze koymak yerine deterministik karar motorunu merkeze almasıdır. Büyük dil modeli yalnızca kararın anlatım katmanı olarak kullanılır.

Temel prensip: Finansal karar önce hesaplanır, sonra anlatılır. Model karar vermez; karar motorunun sonucunu anlaşılır hale getirir.

2. Kullanılan Teknolojiler

Teknoloji	Görevi
TypeScript	Tip güvenliği sağlar, veri şekillerini açık hale getirir.
React	Arayüzü bileşenlere ayırır.

Next.js	Sayfalar, API route'lar ve server rendering altyapısını sağlar.
Tailwind CSS	Responsive ve hızlı arayüz geliştirme sağlar.
Supabase	Auth, PostgreSQL ve server-side veri erişimi sağlar.
Resend	Email doğrulama kodlarını gönderir.
SerpAPI	Google News ve Google Shopping verilerini getirir.
Gemini API	Yerel karar sonucunu doğal Türkçe cevaba dönüştürür.
Frankfurter API	Döviz kuru dönüşümü yapar.
Vercel	Deployment, environment variable ve cron altyapısı sağlar.

3. Sistem Mimarisi

Sarowth üç ana katmandan oluşur: arayüz katmanı, uygulama/karar katmanı ve veri katmanı.

- Arayüz katmanı: React bileşenleri, formlar, grafikler, chat ve panel ekranları.
- Uygulama katmanı: API route'lar, server action'lar, karar motoru, agent endpointleri.
- Veri katmanı: Supabase PostgreSQL tabloları ve auth sessionları.

Chat veri akışı

1. Kullanıcı mesaj gönderir.
2. `/api/assistant` endpointi mesajı alır.
3. Supabase session doğrulanır.
4. Bütçe ve agent verileri çekilir.
5. Komut ayrıştırılır.
6. Gerekirse SerpAPI veya kur API'si çağrılır.
7. Yerel karar motoru sonucu üretir.
8. Gemini anlatım katmanı olarak çağrılır.
9. Cevap ve dashboard verisi client'a döner.

4. Veritabanı Tasarımı

Veritabanı tasarımında veriler ayrı sorumluluklara göre tablolara bölünür. Kullanıcı profili, bütçe kayıtları, chat geçmişi, haberler ve tedarik linkleri aynı tabloda tutulmaz.

Tablo	Amaç
profiles	Kullanıcı profil, gelir, tasarruf hedefi ve risk tercihi.
budget_entries	Gelir, gider ve birikim kayıtları.
assistant_messages	Chat geçmişi.
agent_watch_topics	Takip edilen ürün ve haber konuları.
market_product_signals	Ürün trend sinyalleri.
finance_news_items	Haber sonuçları.
product_supplier_links	Ürün ve tedarik linkleri.
agent_runs	Agent çalışma logları.

5. Kimlik Doğrulama

Proje email ve 6 haneli doğrulama kodu kullanır. Kullanıcı email adresini girer, sistem kod üretir, kod hashlenir, Resend ile gönderilir ve kullanıcı kodu doğruladığında Supabase session oluşturulur.

Client tarafında yapılan doğrulama güvenlik için yeterli değildir. Kritik kontroller her zaman server tarafında da yapılmalıdır.

6. Bütçe Modülü

Bütçe modülü karar motorunun temelidir. Kullanıcı kayıt türünü seçer: gelir, gider veya birikim. Her tür için ayrı kategori listesi gösterilir. Bu, verinin temiz kalmasını sağlar.

Server action kayıt alırken kategori ile kayıt türünün uyumlu olup olmadığını kontrol eder. Böylece kullanıcı doğrudan request gönderse bile hatalı veri yazılamaz.

7. Karar Motoru

Satın alma karar motoru kullanıcının gelirini, giderini, birikimini ve serbest bütçesini hesaplar. Kullanıcı bir tutar verdiğinde sistem bu tutarın bütçeyi ne kadar zorladığını ölçer.

- **ALINABİLİR** : Satın alma sonrası güvenli alan korunur.
- **BEKLE** : Satın alma mümkün görünür ama bütçeyi zorlar.
- **ALMA** : Satın alma güvenli alanın dışına çıkarır.

Bu karar motoru deterministiktir. Aynı veriyle aynı sonucu üretir. Finansal karar desteği için bu tutarlılık önemlidir.

8. Agent Mimarisi

Agent, belirli bir hedef için veri toplayan, filtreleyen, skorlayan ve sisteme yazan görev birimidir. Sarowth'ta agentlar sadece chat cevabı üretmez; haber, ürün ve tedarik verisini panele bağlar.

- Haber agentı: Global haberleri arar.
- Ürün agentı: Yerel veya global ürün sonuçlarını arar.
- Intelligence agent: Takip edilen konuları periyodik işler.
- Market agent: Trend ürün sinyallerini günceller.

9. Haber Arama Algoritması

Haber aramasında kullanıcının yazdığı cümleden ana konu çıkarılır. Örneğin **tesla hissesi son durum** ifadesinde ana konu **tesla** olur. **hisse**, **son durum**, **haber** gibi kelimeler arama niyetidir, ana konu değildir.

1. Metin normalize edilir.
2. Gereksiz haber kelimeleri çıkarılır.
3. Ana konu belirlenir.
4. Global sorgu varyasyonları üretilir.
5. SerpAPI Google News çağrılır.
6. Sonuçlar fuzzy eşleşme ile filtrelenir.
7. Skora göre sıralanır.

10. Ürün Arama Algoritması

Ürün aramasında genel kategori sonucu yeterli değildir. Kullanıcı marka ve model girdiyse sonuç başlığında bu tokenların anlamlı şekilde eşleşmesi gerekir.

Örneğin `kaabo wolf warrior x scooter` araması sadece scooter sonuçlarıyla doldurulmamalıdır. Kaabo, Wolf, Warrior ve X bilgileri ürünün kimliğini oluşturur.

Kullanıcı ürün adının sonuna `global` eklerse global pazar aranır. Aksi halde kullanıcının konumuna göre yerel pazar aranır.

11. Fuzzy Eşleşme

Fuzzy eşleşme, küçük yazım hatalarına tolerans tanır. Metin normalize edilir, tokenlara ayrılır ve sonuç başlığıyla karşılaştırılır. Edit distance kullanılarak benzer kelimeler kabul edilebilir.

Bu teknik hem eksik sonuç sorununu hem de alakasız sonuç sorununu azaltır. Ancak eşik değerleri dikkatli seçilmelidir. Çok gevşek eşik alakasız sonuç getirir; çok sıkı eşik doğru sonuçları kaçıır.

12. Yenileme ve Tekrar Önleme

Kullanıcı haberleri veya ürünleri yenilediğinde aynı sonuçların tekrar gösterilmemesi gerekir. Bu yüzden client, ekranda görünen başlık ve URL listesini server'a gönderir. Server bu sonuçları dışlar.

URL'ler normalize edilir. Başlıklar da normalize edilir ve token benzerliğiyle karşılaştırılır. Aynı haber farklı URL ile gelse bile başlık benzerliğinden elenmeye çalışılır.

13. Para Birimi ve Konum

Ürün araması kullanıcının konumuna göre yerel sonuç getirebilir. Konum bilgisi Vercel veya Cloudflare headerlarından okunur. Eğer global arama seçilmişse global pazar aranır ama fiyat kullanıcının yerel para birimine çevrilmeye çalışılır.

Kur dönüşümü başarısız olursa sistem orijinal fiyatı korur. Bu, kullanıcıya yanlış dönüşüm göstermekten daha güvenlidir.

14. Sıfırdan Geliştirme Planı

1. Next.js TypeScript projesi oluştur.
2. Tailwind CSS kur.
3. Supabase projesi aç.
4. Veritabanı tablolarını oluştur.
5. Email doğrulama akışını yaz.
6. Bütçe formunu geliştir.
7. Bütçe özet fonksiyonunu yaz.
8. Satın alma karar motorunu yaz.
9. Chat UI oluştur.
10. Assistant API route yaz.
11. SerpAPI haber araması ekle.
12. SerpAPI ürün araması ekle.
13. Fuzzy eşleşme algoritmasını yaz.
14. Gemini anlatım katmanını ekle.
15. Agent sonuçlarını Supabase'e kaydet.
16. Dashboard panellerini bağla.
17. Yenileme ve tekrar önleme mantığını ekle.
18. Vercel deployment yap.

15. Kod Okuma Sırası

1. `README.md`
2. `supabase/schema.sql`
3. `app/workspace/actions.ts`
4. `components/BudgetEntryForm.tsx`
5. `app/page.tsx`
6. `components/AssistantChat.tsx`
7. `components/AgentIntelligenceWorkspace.tsx`
8. `app/api/assistant/route.ts`
9. `app/api/agents/intelligence/route.ts`

10. `lib/agents/security.ts`

16. Profesyonel Geliştirme Kuralları

- Her fonksiyon tek sorumluluk taşımalıdır.
- Client doğrulaması server doğrulamasının yerine geçmemelidir.
- API key asla client tarafına çıkmamalıdır.
- Dış API sonuçları filtrelenmeden kullanıcıya gösterilmemelidir.
- Finansal kararlar yalnızca LLM'e bırakılmamalıdır.
- Her önemli değişiklikten sonra build alınmalıdır.
- Veritabanı şeması ve ortam değişkenleri dokümante edilmelidir.

Sonuç

Sarowth, kişisel finans verisini canlı haber ve ürün agentlarıyla birleştiren, kontrollü yapay zeka anlatım katmanına sahip bir karar destek sistemidir. Bu el kitabı takip edilerek proje sıfırdan geliştirilebilir ve geliştirici modern web mimarisi, veri modelleme, API entegrasyonu, agent mantığı, fuzzy arama ve güvenli karar motoru tasarlamayı öğrenebilir.